

Rapport de Stage

Développement d'outils Python pour DFTS-P2MINER



Laboratoire d'InfoRmatique en Image et Systèmes d'information (LIRIS)
Équipe Imagine

Université Claude Bernard  Lyon 1

Université Claude Bernard Lyon 1
Licence Informatique L3 – Année 2024/2025

Étudiant : BYTYQI Leon

Numéro étudiant :

Tuteur de stage : Alexandre Meyer

Période de stage : Du 2 juin au 25 juillet 2025

Organisme d'accueil : Laboratoire d'InfoRmatique en Image et Systèmes d'information (LIRIS)

Équipe : Imagine

Tutrice entreprise : Catherine Pothier

Table des matières

Rapport de Stage	0
Table des matières.....	1
1 – Remerciements	2
2 – Résumé.....	2
3 – Introduction	3
4 – Présentation du LIRIS	3
5 - Présentation de l'équipe Imagine	4
6 - Présentation du projet DFTS-P2MINER	4
7 - Conditions et objectifs du stage.....	4
8 - Travail réalisé.....	5
Préambule : présentation de DFTS-P2MINER et du cadre de travail	5
8.1 - Préparation de l'environnement de travail.....	6
8.2 - Pré-traitement des données.....	6
8.3 - Développement d'un afficheur pour les données brutes	7
8.4 - Recadrage géographique et optimisation de la zone de travail	8
8.5 - Tests multiples et agrégation spatiale pour optimiser le temps de traitement.....	9
8.6 Automatisation du calcul des paramètres sigma pour DFTS-P2MINER	10
8.7 - Automatisation de l'analyse visuelle et génération d'exports pour Google Earth.....	11
8.8 - Analyse du lien entre le paramètre sigma et la détection de motifs.....	12
8.9 - Superposition de patterns et intégration sur fonds cartographiques.....	13
8.10 - Projet de recherche : corrélation inter-exécutions et analyse temporelle avancée	14
9 – Difficultés rencontrées et solutions apportées	16
10 – Conclusion	17
11 – Documentation et liens	18

1 – Remerciements

Je tiens avant tout à exprimer ma profonde gratitude à toute l'équipe du laboratoire LIRIS et plus particulièrement à l'équipe Imagine pour l'accueil chaleureux qu'elle m'a réservé dès mon arrivée. Leur bienveillance, leur disponibilité et l'enthousiasme qui règne dans les locaux ont créé un cadre de travail à la fois stimulant et enrichissant.

Je remercie tout particulièrement Madame Catherine Pothier, maître de conférences à l'INSA et membre de l'équipe Imagine, qui a été ma tutrice de stage et mon guide au quotidien. Par ses conseils avisés, sa patience et son expertise dans le domaine de la fouille de données, elle m'a permis d'appréhender les enjeux techniques du projet DFTS-P2MINER. Ses retours constructifs m'ont encouragé à développer mes compétences en programmation Python et en traitement de données.

Ma reconnaissance s'adresse également à l'ensemble de l'équipe Imagine, ses doctorants et autres stagiaires, pour l'environnement de recherche stimulant qu'elle offre et m'avoir permis de contribuer à un projet de recherche dans le domaine d'analyse de séries temporelles géospatiales.

Enfin, je souhaite remercier l'Université Claude Bernard Lyon 1 pour m'avoir donné l'opportunité de réaliser ce stage au sein d'un laboratoire de recherche.

2 – Résumé

Ce rapport présente le travail effectué durant mon stage de 8 semaines au sein du laboratoire LIRIS (Laboratoire d'InfoRmatique en Image et Systèmes d'information), dans l'équipe Imagine, du 2 juin au 25 juillet 2025. Le stage s'est déroulé sous la supervision de Catherine Pothier, maître de conférences à l'INSA Lyon.

L'objectif principal de ce stage était de contribuer au développement d'outils Python pour améliorer l'entrée des données et l'exploitation des résultats du prototype DFTS-P2MINER. Ce logiciel, développé par l'équipe de recherche du LIRIS en collaboration avec le LISTIC (Laboratoire d'Informatique, Systèmes, Traitement de l'Information et de la Connaissance) et l'Université de Savoie Mont-Blanc, permet d'extraire et de classer des motifs séquentiels fréquents groupés (GFS-patterns) à partir de séries temporelles de champs de déplacements géospatial.

Les missions principales ont consisté à développer des outils de pré-traitement pour faciliter l'intégration de séries temporelles longues, à implémenter des techniques de fenêtrages glissant, et à concevoir des outils d'analyse automatique des motifs extraits. J'ai également travaillé sur l'amélioration des méthodes de visualisation des résultats et effectué des séries de tests rigoureux sur des jeux de données.

Ce stage m'a permis de consolider mes compétences en programmation Python, d'apprendre le traitement de données images, et de découvrir les applications de la fouille de données.

3 – Introduction

Mon stage s'est déroulé au sein de l'équipe Imagine du laboratoire LIRIS, situé sur le campus de la Doua. Ce laboratoire de recherche, rattaché à l'Université Claude Bernard Lyon 1, l'INSA, l'Université Lyon 2 et également l'Ecole Centrale, rassemble des chercheurs, enseignants-chercheurs, doctorants et stagiaires autour de thématiques liées à l'image, aux données et à l'intelligence artificielle.

J'ai travaillé sur un projet existant, DFTS-P2MINER, un outil de recherche développé dans le cadre de travaux en fouille de données appliquée aux séries temporelles géospatiales. Mon rôle a été d'apporter des améliorations au code, de créer de nouvelles extensions et d'optimiser le flux de traitement des données.

4 – Présentation du LIRIS

Le LIRIS (Laboratoire d'Informatique en Image et Systèmes d'Information) est une unité mixte de recherche associée au CNRS, à l'Université Claude Bernard Lyon 1, à l'INSA Lyon, à l'Université Lumière Lyon 2, à l'École Centrale de Lyon et à l'École Normale Supérieure de Lyon.

Ses domaines de recherche couvrent notamment l'informatique graphique, la vision par ordinateur, la fouille de données, l'interaction homme-machine, et la science des données.

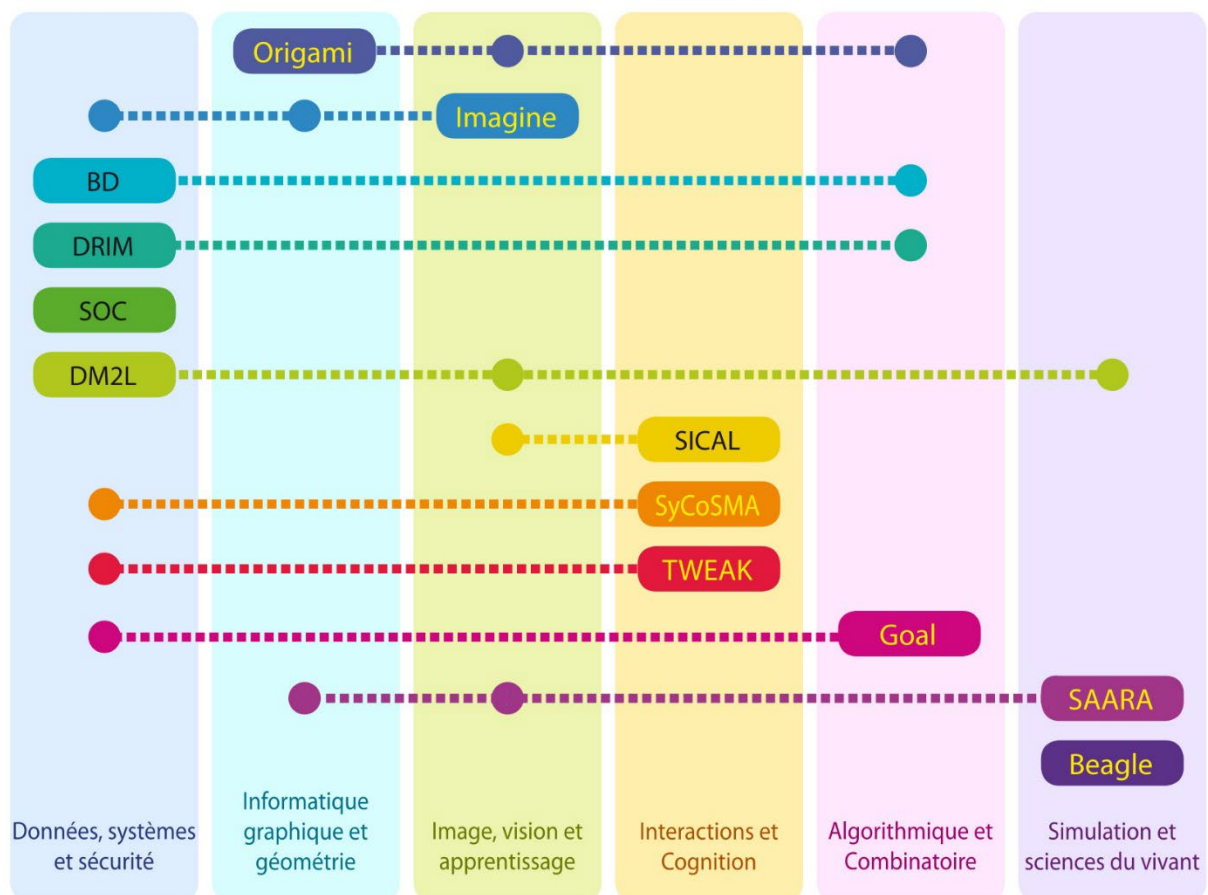


Illustration du pôle de compétence principal

5 - Présentation de l'équipe Imagine

L'équipe Imagine est spécialisée dans l'analyse et le traitement d'images et de données spatiales et temporelles. Elle participe à des projets collaboratifs dans le domaine de l'observation de la Terre, souvent en partenariat avec d'autres laboratoires et agences spatiales.

L'équipe Imagine du LIRIS est spécialisée dans la vision par ordinateur, l'apprentissage et la reconnaissance de formes. Elle réunit **23 membres permanents, 33 doctorants et 6 post-doctorants** et de nombreux stagiaires.

Dans le cadre du projet DFTS-P2MINER, elle s'intéresse à la détection et à l'analyse de motifs récurrents dans des données géophysiques.

6 - Présentation du projet DFTS-P2MINER

DFTS-P2MINER est un outil expérimental conçu pour identifier des motifs séquentiels fréquents groupés (GFS-patterns) dans des séries temporelles représentant des champs de déplacements. Ces données proviennent souvent de mesures satellitaires, comme l'interférométrie radar (InSAR).

Le logiciel comprend plusieurs modules :

- un pré-traitement pour formater et nettoyer les données
- un moteur de détection basé sur l'algorithme P2MINER
- un module de post-traitement pour visualiser et analyser les motifs détectés

7 - Conditions et objectifs du stage

Le stage s'est déroulé sur site, dans les locaux du LIRIS, en collaboration étroite avec ma tutrice et les membres de l'équipe.

Objectifs initiaux :

- Développer des scripts Python pour faciliter le pré-traitement des données
- Améliorer l'intégration de longues séries temporelles
- Mettre en place des outils d'analyse et de visualisation automatiques

8 - Travail réalisé

Préambule : présentation de DFTS-P2MINER et du cadre de travail

Avant de détailler les missions effectuées, il est important de présenter l'outil central de ce stage : DFTS-P2MINER.

Ce logiciel, développé par DM2L et l'équipe Imagine du LIRIS en collaboration avec d'autres laboratoires, est un prototype de recherche conçu pour détecter et analyser des **motifs séquentiels fréquents groupés** (*GFS-patterns*) dans des séries temporelles de données géospatiales.

À quoi sert DFTS-P2MINER

L'outil s'inscrit dans des travaux de recherche liés à l'observation de la Terre, notamment via des données satellites. Il permet d'identifier des motifs récurrents dans le temps et l'espace, par exemple des mouvements de terrain, des déformations ou autres phénomènes mesurables. Ces informations sont précieuses pour les géoscientifiques et spécialistes de télédétection, car elles aident à comprendre l'évolution de zones d'intérêt et à anticiper certains changements.

Les données en entrée

Les données traitées par DFTS-P2MINER proviennent principalement de mesures satellitaires obtenues par interférométrie radar (**InSAR**).

Elles se présentent généralement sous forme de cubes multi-bandes GeoTIFF :

- chaque **bande** correspond à une image acquise à une date précise,
- chaque **pixel** contient une valeur représentant un déplacement ou une variation physique à cet endroit.

Ces fichiers peuvent être très volumineux et couvrir de larges zones géographiques. Une étape de pré-traitement est donc indispensable pour sélectionner la zone d'intérêt, nettoyer les données et les mettre au format attendu par le logiciel.

Le fichier de paramètres

Pour lancer une analyse, DFTS-P2MINER utilise un fichier de configuration dans lequel on définit :

- la liste et l'ordre des images à analyser,
- les dates associées à chaque image,
- les valeurs de *sigma* (pourcentage minimal de surface recouverte par un motif) à tester,
- et d'autres paramètres comme la taille des fenêtres, le nombre minimal d'occurrences, ou les filtres.

Le bon réglage de ces paramètres est crucial, car il conditionne la pertinence et la qualité des motifs détectés.

Les sorties produites

En sortie, DFTS-P2MINER génère :

- des **images de résultats** où les motifs sont colorés, chaque couleur étant associée à une date,
- des fichiers **logs** détaillant les motifs et leurs caractéristiques,
- et, selon la configuration, des images intermédiaires (*firstImage*, *intermediateImage*, *lastImage*) qui permettent de retracer la chronologie d'apparition d'un motif.

Ces résultats bruts nécessitent un travail d'interprétation et de validation, ce qui a motivé le développement, au cours de mon stage, de nombreux outils d'automatisation et de visualisation pour en faciliter l'exploitation scientifique.

8.1 - Préparation de l'environnement de travail

Dès les premiers jours, une partie importante de mon temps a été consacrée à préparer un environnement de travail. L'outil DFTS-P2MINER manipulant de gros volumes de données, il était nécessaire d'avoir une machine correctement configurée.

L'ordinateur mis à disposition au laboratoire a été configuré en dualboot Linux/Windows.

L'installation a impliqué :

- le partitionnement du disque et la configuration pour le démarrage,
- l'installation des distributions nécessaires (Ubuntu côté Linux),
- la configuration des environnements Python via conda pour isoler les dépendances du projet.

Pour assurer la reproductibilité, j'ai documenté chaque étape d'installation dans un fichier Markdown partagé avec l'équipe, afin qu'un futur stagiaire ou membre puisse répliquer l'environnement en quelques commandes.

8.2 - Pré-traitement des données

Ma première mission consistait à préparer les données géophysiques brutes afin qu'elles puissent être exploitées par DFTS-P2MINER. Les fichiers que je recevais étaient des GeoTIFF multi-bandes de grande taille, des "cubes" d'informations dans lesquels toutes les données temporelles et spatiales étaient compilées ensemble. Chaque bande correspondait à un instant précis de la série temporelle, mais tant que tout restait compressé, il était impossible de traiter ces informations.

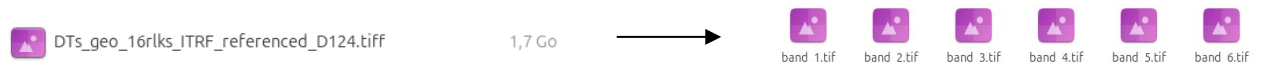
Pour répondre à ce problème, j'ai développé un script Python capable de découper ces cubes bande par bande. En utilisant la librairie **GDAL**, j'ai pu :

- ouvrir le fichier source et analyser sa structure,
- extraire chaque bande séparément,

- sauvegarder chaque image au format TIFF 2D, tout en conservant les métadonnées géospatiales (projection, système de coordonnées, géotransformation).

Une autre étape essentielle a été la gestion des valeurs manquantes. Les fichiers bruts contenaient souvent des zones marquées par la valeur *NaN*, indiquant une absence de mesure. Par convention dans le projet, ces valeurs devaient être remplacées par -1 000 000 pour ne pas perturber les traitements ultérieurs. J'ai donc intégré cette conversion directement dans le script, ce qui garantissait que chaque image extraite soit propre et prête à être utilisée.

En sortie, le gros fichier initial se transformait en un dossier parfaitement organisé, contenant une image par bande numérotée dans l'ordre, avec toutes ses métadonnées préservées. Ce travail a eu un impact important sur la suite du projet : il a rendu les données bien plus faciles à manipuler, a réduit les temps de traitement et a assuré une compatibilité totale avec les étapes suivantes du pipeline.



8.3 - Développement d'un afficheur pour les données brutes

Après avoir mis en place le découpage des cubes multi-bandes en images individuelles, il était important de pouvoir visualiser rapidement les données, que ce soit directement à partir du cube original ou à partir des bandes extraites. Cette visualisation permettait de localiser la zone couverte par le jeu de données et d'évaluer immédiatement sa pertinence pour les traitements à venir.

Pour répondre à ce besoin, j'ai développé un afficheur Python capable de lire une bande GeoTIFF et de l'afficher en conservant toutes les informations essentielles. Cet outil se connecte au fichier choisi, remplace les valeurs conventionnelles -1 000 000 par *NaN* pour éviter les perturbations visuelles, puis extrait et affiche :

- les dimensions de l'image,
- la résolution spatiale,
- le nombre total de pixels et le nombre de pixels *NaN*,
- les coordonnées géographiques exactes du coin nord-ouest et du coin sud-est.

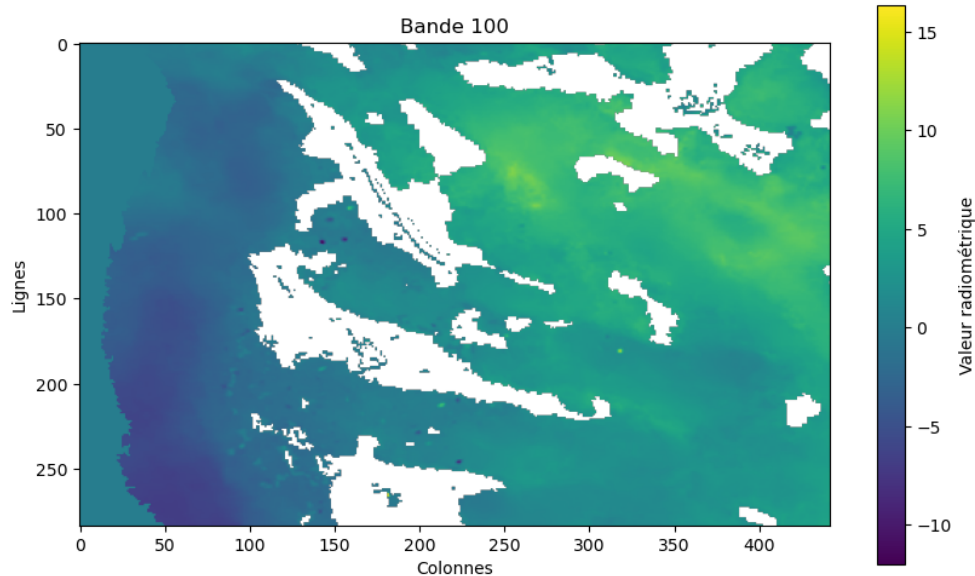
Ces informations s'affichent d'abord dans la console, ce qui permet d'obtenir rapidement un diagnostic chiffré sur l'état de la bande. Dans un second temps, l'afficheur ouvre une fenêtre **Matplotlib** présentant l'image, avec une échelle graduée et un code couleur reflétant les valeurs radiométriques.

```

=====
Date d'acquisition : Non disponible
Dimensions : 442x284
Résolution : 0.001111° x -0.001111°
Pixels totaux : 125528
Pixels NaN : 0

Coordonnées géographiques :
NW: 14.865972°E, 44.989531°N
SE: 15.357084°E, 44.673975°N
=====

```



Représentation du résultat obtenu

Cet outil m'a été particulièrement utile pour valider la cohérence des bandes après leur extraction. Par exemple, il permettait de repérer immédiatement si une bande contenait une proportion anormale de pixels manquants ou si la zone géographique ne correspondait pas aux attentes. Cela m'a évité de lancer des traitements lourds sur des données inutilisables.

À terme, cet afficheur est devenu un élément essentiel du flux de travail, servant non seulement à vérifier la qualité des données mais aussi à documenter visuellement l'avancée des travaux. Il sera illustré dans ce rapport par une capture d'écran montrant l'interface et la sortie graphique générée.

8.4 - Recadrage géographique et optimisation de la zone de travail

Même si les données que nous recevions avaient déjà subi un premier traitement par les équipes de recherche, elles restaient dans la majorité des cas très volumineuses et couvraient une zone géographique bien plus large que celle réellement nécessaire à notre étude. Or, dans notre cadre, analyser un cube complet de cette taille aurait demandé des temps d'exécution considérables, pour un résultat dont une grande partie ne concernait pas notre zone d'intérêt.

Avec ma tutrice, nous définissions donc à l'avance une zone précise sur laquelle concentrer le travail. La sélection se faisait en visualisant les coordonnées d'origine du cube dans Google Earth, puis en délimitant ensemble le rectangle correspondant à notre aire d'étude. Une fois la zone fixée, je passais à la modification du fichier source.

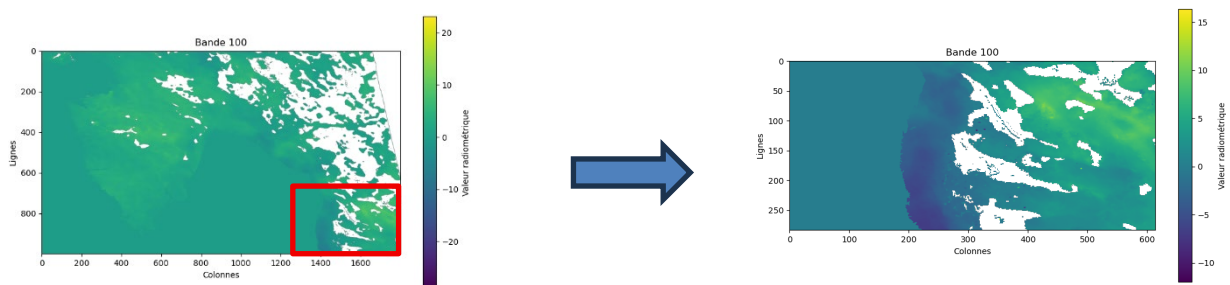
J'ai développé un script Python, toujours basé sur la librairie **GDAL**, qui permettait de recadrer le cube multi-bandes. Ce script modifiait directement la géoréférence du fichier, en remplaçant les

coordonnées du coin nord-ouest (NW) et du coin sud-est (SE) par celles correspondant à notre nouvelle zone. L'avantage était double :

- réduire le volume de données traité, donc accélérer considérablement les traitements
- concentrer l'analyse sur la zone réellement pertinente pour notre projet

Afin de préserver la lisibilité visuelle, je n'ai pas simplement tronqué les images : j'ai aussi ajusté la taille des pixels de manière proportionnelle, ce qui permettait de conserver une représentation correcte à l'affichage.

En sortie, nous obtenions un nouveau cube, plus petit, parfaitement géoréférencé, et prêt à passer dans la chaîne de traitement DFTS-P2MINER. Ce recadrage, que nous appliquons systématiquement, a été l'une des optimisations les plus importantes pour rendre nos expérimentations plus rapides et ciblées.



Exemple de recadrage géographique

8.5 - Tests multiples et agrégation spatiale pour optimiser le temps de traitement

Un aspect marquant de mon stage a été le nombre considérable de tests réalisés avec DFTS-P2MINER. Comme précisé dans le préambule, j'ai passé une grande partie de mon temps à lancer des traitements successifs afin d'évaluer l'influence de différents paramètres sur la détection des motifs. Je testais notamment la variation du pourcentage de surface recouverte, en allant de 2 à 20 % par pas de 2 %, ainsi que le nombre d'images fournies en entrée. Dans la logique du projet, plus il y avait d'images, plus les résultats étaient précis, mais aussi plus le temps de traitement augmentait.

Malgré le recadrage géographique des données, certains jeux restaient extrêmement volumineux. Sur les zones dépassant des dimensions d'environ 900 x 900 pixels, le traitement complet pouvait devenir très long, parfois au point de bloquer l'avancée des analyses. Avec ma tutrice, nous avons donc cherché une solution pour réduire la taille des données tout en conservant une qualité exploitable.

La méthode retenue a été l'**agrégation spatiale** : au lieu de traiter chaque pixel individuellement, nous regroupions les données par blocs de taille fixe (2 x 2 le plus souvent, mais plus si nécessaire) et remplacions chaque bloc par la moyenne de ses valeurs. Cette moyenne est une moyenne simple, et non pondérée : chaque pixel du bloc contribue également au calcul, à l'exception des valeurs marquées comme *NoData*, qui sont ignorées.

Ce traitement avait plusieurs avantages :

- il réduisait la taille du jeu de données et donc le **temps d'exécution**, souvent de moitié
- il conservait la cohérence visuelle et spatiale des données
- il minimisait la perte d'information significative, surtout sur les zones où les variations étaient faibles

En pratique, cette étape intervenait juste après le recadrage. Le script parcourait chaque bande du cube, découpait l'image en blocs carrés, calculait la moyenne de chaque bloc en ignorant les valeurs *NoData*, puis reconstruisait un nouveau cube avec une résolution réduite. Les métadonnées géographiques étaient mises à jour pour refléter cette nouvelle résolution, assurant ainsi la compatibilité avec le reste du pipeline.

Grâce à cette optimisation, nous avons pu traiter rapidement même les jeux de données initialement trop grands, comme ceux atteignant 1300 x 1000 pixels, tout en maintenant une précision suffisante pour les besoins de nos analyses.

8.6 Automatisation du calcul des paramètres sigma pour DFTS-P2MINER

Une fois le pré-traitement terminé et les données prêtes, la phase suivante consistait à préparer l'exécution de DFTS-P2MINER dans les meilleures conditions. L'un des paramètres essentiels à ajuster est la valeur de **sigma**, qui détermine la surface minimale recouverte par les motifs détectés. Le choix de cette valeur influence directement le nombre et la qualité des patterns extraits, et il est souvent nécessaire de tester toute une plage de valeurs pour identifier celle qui offre le meilleur compromis entre précision et volume de résultats.

Pour éviter d'effectuer ces calculs manuellement à chaque nouvelle série de tests, j'ai développé un script Python qui automatise la génération des paramètres sigma. L'idée est de se baser sur le nombre moyen de pixels manquants (*NaN*) présents dans les bandes traitées par DFTS, afin de calculer, pour chaque pourcentage de surface recouverte que nous voulons tester, le seuil numérique correspondant.

Le script commence par parcourir une liste prédéfinie de bandes d'intérêt. Pour chacune, il compte le nombre de pixels *NaN*, puis calcule la moyenne de ces valeurs sur l'ensemble des bandes. À partir de cette moyenne, il génère un fichier listant les couples *facteur_sigma* / *seuil_sigma* correspondant à différents pourcentages de surface :

- d'abord 0,5 % et 1 %
- puis de 2 % à 20 %, par pas de 2 %

Par exemple, si la moyenne de pixels manquants par bande est de 5 000, et que l'on souhaite tester un sigma correspondant à 10 % de surface recouverte, le script calcule directement un seuil de 500 pixels à passer à DFTS-P2MINER.

Ce fichier de paramètres est ensuite directement exploitable par les scripts de lancement du traitement. Cette automatisation nous a permis de gagner un temps précieux, surtout dans le cadre de campagnes de tests intensifs où nous enchaînions des dizaines d'exécutions avec des configurations différentes.

En pratique, cette étape s'insérerait juste avant l'exécution du moteur DFTS, garantissant que chaque série de traitements disposait de valeurs sigma adaptées aux données analysées, plutôt que de reposer sur des estimations approximatives.

8.7 - Automatisation de l'analyse visuelle et génération d'exports pour Google Earth

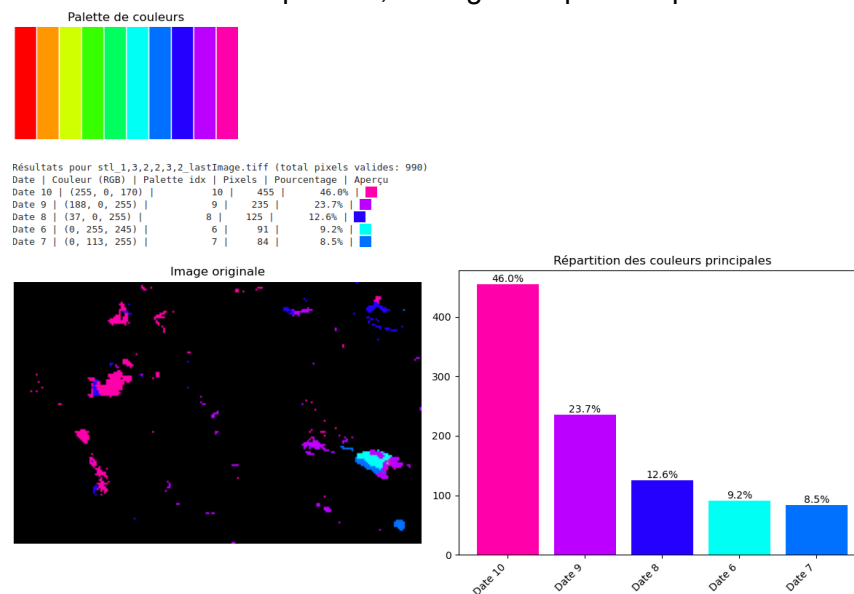
Une fois DFTS-P2MINER exécuté, le logiciel produit un ensemble d'images de résultats dans lesquelles chaque motif détecté est codé par couleur. Ces images, bien que riches en informations, ne sont pas immédiatement exploitables pour une analyse scientifique précise : il faut en extraire des statistiques claires et les relier aux dates d'acquisition des données.

Pour faciliter ce travail, j'ai développé un script Python d'analyse automatisée des résultats. Celui-ci prend en entrée une image issue de DFTS et commence par identifier la palette de couleurs spécifique associée à ce jeu de données. Chaque couleur correspond à une date dans la série temporelle, ordonnée de la première à la dernière, de droite à gauche dans la palette.

À partir de là, le script effectue :

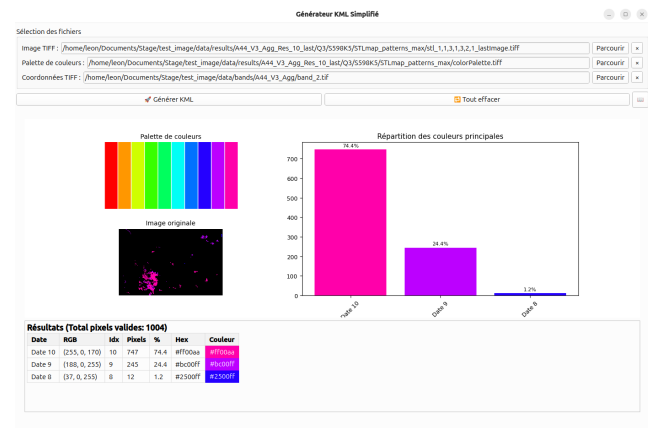
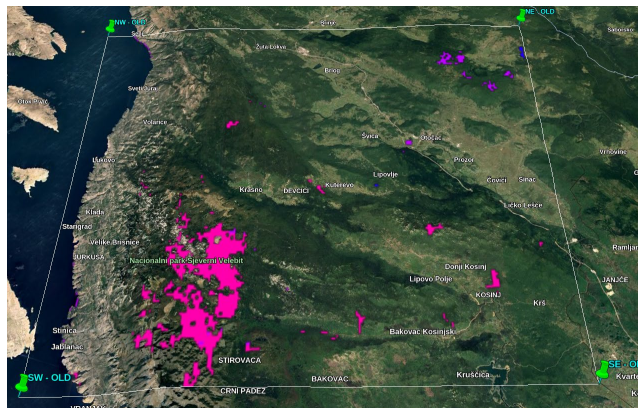
- le **comptage du nombre de pixels** pour chaque couleur,
- le calcul de leur **répartition en pourcentage**,
- la production d'un **graphique détaillé** illustrant la fréquence relative des couleurs principales,
- l'association automatique de chaque couleur à la date correspondante.

L'objectif était que les équipes scientifiques puissent, en un coup d'œil, savoir quand un motif est apparu et quelle proportion de la zone il couvre. L'outil gère aussi les exclusions : les pixels noirs ou blancs, ou ceux totalement transparents, sont ignorés pour ne pas fausser les calculs.



Présentation de l'analyse visuelle extraite

En complément de cette version console et graphique (Matplotlib), j'ai conçu une version enrichie en interface **PyQt6**. Celle-ci permet de sélectionner facilement l'image à analyser et de retrouver toutes les informations directement dans une fenêtre interactive. Mais surtout, cette interface ajoute une fonctionnalité clé : la génération automatique d'un fichier PNG géoréférencé à partir de l'image analysée, utilisable pour produire un fichier **KML**. Ce KML est ensuite ouvert automatiquement dans Google Earth et superposé à la bonne position géographique, ce qui permet aux chercheurs de visualiser le motif dans son contexte réel et de mieux comprendre pourquoi il a été détecté.



Présentation de la version graphique ainsi que integration sur Google Earth

Ce script a eu un impact immédiat sur le workflow : là où auparavant l'interprétation des résultats pouvait prendre plusieurs minutes par image, elle est devenue quasi instantanée. Les équipes peuvent désormais enchaîner rapidement les analyses et se concentrer sur l'interprétation scientifique des motifs, plutôt que sur la manipulation fastidieuse des fichiers.

8.8 - Analyse du lien entre le paramètre sigma et la détection de motifs

Dans le but d'optimiser la recherche de motifs significatifs, il ne suffit pas de lancer DFTS-P2MINER avec des paramètres arbitraires : il faut identifier la configuration la plus efficace pour extraire un maximum de patterns pertinents. Parmi ces paramètres, le pourcentage de surface recouverte, appelé *sigma*, joue un rôle central.

L'approche adoptée consistait à tester systématiquement une plage de valeurs de 0,5 % à 20 %, par pas de 2 %, puis à analyser l'impact de chacune sur le nombre total de motifs détectés. Le but était simple : repérer le sigma qui génère le plus de patterns, puis concentrer nos jeux de tests sur cette valeur pour affiner l'analyse.

Pour automatiser cette étape, j'ai développé un script Python capable de parcourir l'arborescence complète des résultats produits par DFTS-P2MINER. Dans chaque sous-dossier correspondant à un paramètre sigma (identifié dans le nom par la convention SxxK5), le script recherche le fichier de log contenant le nombre de motifs détectés. Il extrait ensuite cette valeur et l'associe au sigma correspondant.

Une fois toutes les données collectées, elles sont triées et représentées graphiquement à l'aide de **Matplotlib**. Ce graphique met en évidence la relation entre sigma et nombre de motifs détectés, permettant ainsi de visualiser d'un coup d'œil la zone optimale. En complément, le script

produit également un tableau récapitulatif des couples *sigma / nombre de patterns* pour référence rapide.

Ce travail a considérablement amélioré notre efficacité : au lieu de naviguer à vue, nous pouvions désormais identifier rapidement la configuration idéale pour un jeu de données donné, et concentrer nos ressources de calcul sur cette plage de valeurs. Cette approche a également facilité la comparaison entre différents jeux de données, en révélant que certaines zones réagissaient mieux à des paramètres spécifiques, ce qui est une information précieuse pour les équipes scientifiques.

8.9 - Superposition de patterns et intégration sur fonds cartographiques

Pour les équipes scientifiques, il ne suffit pas seulement d'analyser les motifs extraits par DFTS-P2MINER ; il faut aussi pouvoir présenter ces résultats de manière claire et impactante, notamment lors de réunions, de présentations ou de conventions. L'objectif est de rendre visibles, sur un support compréhensible par tous, les zones où des patterns significatifs ont été détectés et sur lesquelles il faut concentrer l'attention.

Pour répondre à ce besoin, j'ai développé un script Python de superposition d'images capable de fusionner plusieurs résultats choisis, et, si nécessaire, de les placer sur un fond cartographique représentant la zone étudiée. Cette approche permet de visualiser directement les motifs dans leur contexte géographique réel, ce qui facilite grandement la compréhension pour des interlocuteurs n'ayant pas accès aux outils ou aux formats spécifiques utilisés pendant le traitement.

Le fonctionnement est en deux étapes. D'abord, chaque image de résultat au format TIFF est convertie en PNG avec un fond transparent : tous les pixels noirs sont supprimés pour ne conserver que les zones colorées représentant les patterns. Ensuite, ces images sont superposées les unes aux autres, soit sur un canevas vide, soit sur une image de fond (par exemple une carte détaillée de la zone), afin de créer un visuel unique et synthétique.

Cette fusion se fait en respectant les proportions et l'alignement spatial, de manière à conserver la correspondance exacte avec les coordonnées géographiques originales. Le résultat final est sauvegardé au format TIFF, prêt à être intégré dans un rapport, une présentation ou même à être retravaillé pour d'autres supports.

Grâce à cet outil, il est devenu possible de préparer en quelques minutes un visuel clair, tout en les inscrivant dans leur environnement réel. C'est une fonctionnalité qui a été particulièrement appréciée par les équipes, car elle leur permettait de confronter directement les motifs calculés avec leurs connaissances terrain.

8.10 - Projet de recherche : corrélation inter-exécutions et analyse temporelle avancée

Au fil du stage, nous nous sommes confrontés à une problématique récurrente : malgré toutes les optimisations mises en place, DFTS-P2MINER restait très lent à l'exécution. Ce ralentissement devenait critique lorsque nous souhaitions traiter un grand nombre d'images (dates) en entrée. Dès que nous atteignions environ 26 ou 27 images, le temps de calcul explosait de façon exponentielle, au point que certaines exécutions semblaient ne jamais se terminer.

Nous avons donc deux contraintes :

1. Pouvoir traiter plus de dates pour enrichir les analyses.
2. Réduire les temps d'exécution sans perdre d'information significative.

Avec ma tutrice, nous avons exploré une piste : scinder l'ensemble des dates à étudier en plusieurs exécutions distinctes, tout en conservant un chevauchement partiel pour pouvoir établir des liens entre les résultats.

La solution imaginée

Prenons un exemple concret : supposons que nous ayons 30 dates à analyser.

- Dans le premier jeu, nous en prenons 20, et dans le second jeu, également 20.
- Les 5 dernières dates du premier jeu deviennent les 5 premières du second.
- Les 15 autres dates de chaque jeu restent exclusives à leur série.

Ce chevauchement théorique permettait d'espérer identifier des motifs communs aux deux exécutions et de comprendre leur continuité temporelle, tout en limitant le volume de données à traiter par exécution.

L'approche d'analyse

Une fois les deux exécutions lancées, l'idée était de :

1. Identifier, grâce aux outils d'analyse développés plus tôt, un **pattern intéressant** dans la première exécution.
2. Comprendre sa structure temporelle exacte, en repérant non seulement les dates où il apparaissait, mais aussi les **gaps** (dates sautées où le motif ne se manifestait pas).
3. Construire, à partir de cette information, un **pattern de recherche** intégrant les gaps.
4. Chercher dans la deuxième exécution **tous les motifs** qui correspondent à ce pattern, en respectant la même structure temporelle et les mêmes sauts.

Les subtilités temporelles

Dans DFTS-P2MINER, chaque symbole dans un nom de fichier correspond à une date :

- Dans un fichier ..._lastImage.tiff, le dernier symbole est la dernière date de la série.

- Dans un fichier `..._firstImage.tiff`, le premier symbole est la première date. Il est également possible de spécifier des images intermédiaires (*intermediateImage*), ce qui permet de reconstruire toute la chronologie d'un motif.

Les gaps ajoutent une difficulté : si un motif n'apparaît pas à certaines dates ou apparaît sur une image bruitée inutile, ces étapes peuvent être ignorées dans la séquence. L'algorithme devait donc être capable de les détecter et de les intégrer dans le modèle de recherche.

Le script développé

Pour automatiser cette analyse complexe, j'ai créé un script capable de :

- Lire une image de référence issue de la première exécution.
- Extraire la palette de couleurs pour relier chaque couleur à une date.
- Reconstituer la séquence temporelle réelle du motif, en analysant directement les pixels des images *first*, *intermediate* et *last*.
- Déterminer le vecteur de gaps correspondant à cette séquence.
- Construire automatiquement un pattern de recherche basé sur les derniers symboles et gaps inclus.
- Parcourir l'ensemble des résultats de la deuxième exécution pour trouver uniquement les motifs respectant à la fois le pattern logique et les gaps détectés.
- Exporter les correspondances validées sous forme de fichiers CSV, de logs détaillés et de copies des images trouvées, afin de faciliter l'analyse manuelle.

Résultats

Ce projet m'a demandé un travail de recherche important, alternant phases théoriques (modélisation sur tableau, définition des règles de correspondance) et phases pratiques (jeux de tests multiples). Il m'a permis de :

- Approfondir ma compréhension des corrélations temporelles dans les données spatio-temporelles.
- Développer une méthodologie pour détecter des motifs communs sur plusieurs exécutions.
- Mettre en place une analyse pixel-par-pixel pour valider les correspondances avec précision.

Même si ce projet n'est pas encore finalisé, la fin de mon stage ayant écourté la phase de validation sur un plus grand nombre de jeux de données, une grande partie du travail de conception et de développement a été réalisée. Les bases sont posées pour poursuivre cette recherche et, à terme, disposer d'un outil permettant de relier efficacement plusieurs exécutions de DFTS et d'optimiser l'exploration des patterns.

9 – Difficultés rencontrées et solutions apportées

Comme dans tout projet technique de recherche, ce stage a été jalonné de difficultés, parfois mineures, parfois plus lourdes, mais qui ont toujours représenté des opportunités d'apprentissage.

La première, que je qualifierais davantage de challenge que de réelle difficulté, fut la familiarisation avec l'ensemble des outils mis à ma disposition. Python et ses bibliothèques spécialisées pour le traitement d'images (GDAL, NumPy, Matplotlib, PIL, etc.) constituaient mon environnement principal. Malgré le fait d'avoir déjà eu une introduction au data mining à travers l'UE « Initiation à l'analyse et synthèse d'images », cette fois-ci il fallait aller plus loin c'est-à-dire mettre les mains dans un projet concret. J'ai donc passé un temps à explorer la documentation, tester des exemples simples, et bâtir mes propres scripts pas à pas. Ce travail de ressourcement m'a permis de prendre confiance et de construire ma boîte à outils personnelle que j'ai réutilisée tout au long du stage.

Une seconde difficulté a concerné la puissance de calcul. L'outil DFTS est particulièrement exigeant en ressources, et très vite ma machine locale a montré ses limites. Sur des jeux de 4 images, l'exécution restait tolérable, mais dès que je passais à 10 images, les temps d'attente devenaient décourageants. Quant aux 20 images, cela aurait été tout simplement impossible. Avec ma tutrice, nous avons donc décidé de migrer sur la plateforme PAGODA, une infrastructure matérielle et logicielle du LIRIS offrant de puissantes ressources de calcul. PAGODA m'a permis de franchir un cap et d'exécuter des jeux de données plus lourds.

Cependant, cette solution a aussi révélé ses limites. Le transfert des fichiers depuis la plateforme vers ma machine était lent, parfois tellement qu'il annulait presque le gain obtenu côté calcul. De plus, l'espace de stockage limité sur PAGODA m'empêchait de conserver les résultats volumineux générés par DFTS. Pour pallier ce problème, nous avons exploré une autre option : le serveur Mac Studio équipé d'une puce M2 Ultra, accessible via AnyDesk. Là, les calculs se sont révélés bien plus rapides (jusqu'à 27 images traitées), et les transferts beaucoup plus fluides. Cette solution m'a donné un vrai confort de travail et m'a permis de continuer mes expérimentations sans perte de temps.

Enfin, la difficulté la plus marquante fut liée au projet de recherche sur l'assemblage des résultats DFTS. Contrairement aux autres missions, ce travail ne consistait pas seulement à appliquer des scripts ou à optimiser un pipeline, il s'agissait de réfléchir à une problématique encore ouverte. Autrement dit, il n'y avait pas de "mode d'emploi" déjà existant. Je suis donc parti de zéro, en travaillant d'abord de manière théorique, crayon à la main et tableau blanc en face, pour imaginer comment découper des séquences temporelles, comment relier deux exécutions différentes, et comment vérifier la validité de correspondances entre patterns. Ce travail de formalisation en amont m'a demandé beaucoup de temps et de tâtonnements, mais il a été extrêmement formateur : j'ai appris à raisonner comme un chercheur, à transformer une intuition en hypothèse testable, et à traduire ensuite cette hypothèse en algorithmes concrets. Même si ce projet reste en partie inachevé à la fin de mon stage, j'ai le sentiment d'avoir posé des bases solides sur lesquelles d'autres pourront s'appuyer.

10 – Conclusion

Ce stage au sein de l'équipe Imagine du laboratoire LIRIS m'a offert une immersion complète dans un environnement de recherche scientifique, où la rigueur des méthodes se conjugue à la créativité nécessaire pour trouver des solutions efficaces.

En huit semaines, j'ai pu contribuer à toutes les étapes de la chaîne de traitement de DFTS-P2MINER. J'ai commencé par développer un pré-processing, capable de préparer des données géophysiques brutes complexes en fichiers exploitables, tout en optimisant leur taille et leur pertinence spatiale. Ce travail a permis de réduire considérablement les temps de calcul et d'améliorer la qualité des entrées.

Pendant la phase de traitement, j'ai conçu des outils pour automatiser la configuration de DFTS, notamment le calcul précis des paramètres *sigma*, et mené de nombreuses campagnes de tests pour comprendre leur impact. J'ai ainsi mis en évidence l'importance de l'adaptation fine des paramètres aux données traitées.

Côté post-processing, j'ai réalisé plusieurs scripts permettant d'analyser automatiquement les résultats, de visualiser et quantifier les patterns détectés, de générer des exports clairs pour Google Earth, et même de produire des superpositions cartographiques adaptées à la communication scientifique. Ces outils se sont révélés précieux pour transformer des résultats bruts en informations directement exploitables par les chercheurs.

Enfin, j'ai initié un projet de recherche ambitieux visant à corréliser plusieurs exécutions de DFTS afin de contourner ses limitations en temps de calcul et en nombre d'images traitables. Ce travail, bien qu'encore en développement, pose les bases d'une approche innovante pour explorer plus largement et plus efficacement les patterns spatio-temporels.

Sur le plan technique, ce stage m'a permis de renforcer mes compétences en Python, traitement d'images géospatiales, optimisation algorithmique et automatisation de workflows complexes. Sur le plan méthodologique, j'ai appris à structurer des expérimentations, documenter rigoureusement des outils et valider des hypothèses. Enfin, sur le plan humain, j'ai découvert la richesse du travail collaboratif en recherche, la nécessité de bien communiquer ses résultats et l'importance de concevoir des outils qui servent directement les besoins des utilisateurs finaux.

Ce stage a été non seulement formateur mais aussi particulièrement stimulant intellectuellement. Il m'a donné envie de poursuivre dans des environnements où la technique et la recherche s'entrecroisent, et où chaque ligne de code contribue à une meilleure compréhension d'un phénomène complexe.

11 – Documentation et liens

- + Site du **LIRIS** – Laboratoire d’Informatique en Image et Systèmes d’information (<https://liris.cnrs.fr/>)
- + Projet **IMAGINE** (<https://projet.liris.cnrs.fr/ imagine/>)
- + Documentation et présentation du **DFTS P2 Miner** (<https://sites.google.com/view/dfts-p2miner>)
- + Présentation de la plateforme **PAGODA** (<https://liris.cnrs.fr/pagoda>)
- + Documentation officielle de **GDAL** (<https://gdal.org/>)
- + Documentation **NumPy** (<https://numpy.org/doc/>)
- + Documentation **Matplotlib** (<https://matplotlib.org/stable/contents.html>)
- + Documentation **Pillow (PIL)** (<https://pillow.readthedocs.io/en/stable/>)
- + Documentation **PyQt6** (<https://doc.qt.io/qtforpython-6/>)
- + Documentation **Pandas** (<https://pandas.pydata.org/docs/>)
- + Documentation **Jupyter** (<https://docs.jupyter.org/en/latest/>)